Addressing Modes, CPU Organization, and Bus Structures

Week 5 — Where Operands Live, and What Moves Them

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 6, 2026

Where We Left Off (Week 4)

Last week: instructions, on paper

- An instruction has an **opcode** field and one or more **address/operand** fields.
- The **instruction cycle** is fetch → decode → execute, driven by PC → MAR → IR → ...
- Registers already in play: PC (next instruction), IR (current instruction), MAR/MDR (memory address/data), AC or a register file R0 . . Rn−1, SP (stack).

The gap we left open

We said an instruction has an "address field." We never said *how* that field turns into the actual location of the operand — or what hardware inside the CPU actually carries the operand from register to register. That is this week.

Roadmap: Three Questions, One Thread

1. **Addressing modes** — given an instruction's address field, how do we compute the *effective address* (EA) of the real operand?
2. **CPU organization** — what hardware (registers, ALU, buses) actually moves operands around inside the CPU?
3. **Single- vs. multiple-bus design** — how many bus wires do we build, and what do we buy with the extra cost?

Running thread for today

We will trace the *same* instruction, $R1 \leftarrow R2 + R3$, through every hardware design we meet — so "faster" and "more expensive" become cycle counts and wire counts, not adjectives.

Today: Where do operands live, and what moves them?

A Question Every Compiler Answers Millions of Times

Consider compiling a single line of C inside a loop:

```
A[i] = A[i] + 1;
```

Socratic question

The generated instruction is something like `ADD R1, X`. But *where does "X" actually live* — a constant? A fixed memory cell? A pointer? An array slot that moves every loop iteration? The instruction format alone doesn't say. Something else has to.

That "something else" is the **addressing mode** encoded in the instruction.

Definition: Addressing Mode

Addressing mode

An **addressing mode** is the rule, specified by the instruction, for computing the **effective address (EA)** — the actual memory location (or register) holding the operand — from the instruction's address field.

- New notation: EA = the computed address of the real operand.
- Different addressing modes trade off **flexibility** (pointers, arrays, relative jumps) against **cost** (extra memory/bus reads to resolve EA).
- We will meet seven modes; each is motivated by a code idiom you already know [1].

Immediate Addressing — the Operand *Is* the Instruction

Idiom: a literal constant

`ADD R1, #5` ⇒ add the literal value 5 to R1.

- The address field *is* the operand — no memory access needed to fetch it.
- EA does not apply; the value is embedded in the instruction word.
- **Fastest** mode (zero extra bus/memory cycles), but the operand must be known at compile time and fit in the field width.
- Compiler use: loop increments, array bounds, small constants.

Direct (Absolute) Addressing

Idiom: a fixed global variable

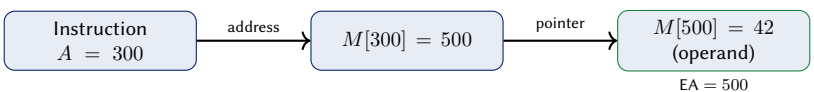
`ADD R1, 1000` $\Rightarrow$ add the contents of memory location 1000 to R1.

- EA $= A$, where A is the address field itself.
- One memory read to fetch the operand from $M[\text{EA}]$.
- Simple, but the address is baked into the instruction — no relocation, no pointers, no arrays without a separate mode.
- Compiler use: statically-allocated global variables.

Indirect Addressing — Pointer Dereference

Idiom: following a pointer

`ADD R1, (1000)` ⇒ `M[1000]` holds *another address*; go there for the operand.



Cost: *two* memory reads (one to get the pointer, one to get the operand).

Register and Register-Indirect Addressing

Register addressing

Idiom: a local variable

```
ADD R1, R2
```

- Operand is already *in* a register — no memory access at all.
- Cheapest mode after immediate.

Register-indirect addressing

Idiom: a pointer variable

```
ADD R1, (R2)
```

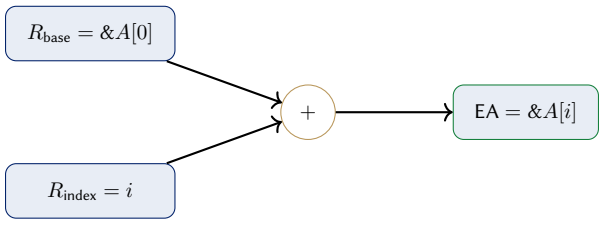
- EA $=$ [R2]: the register *holds an address*.
- One memory read to fetch M[EA].

Compare: register-indirect is indirect addressing's cheaper cousin — the pointer lives in a register instead of memory, so it costs one memory read instead of two.

Displacement Addressing — How Arrays Actually Work

Family: relative, based, indexed

EA = [register] + displacement



This is exactly how `A[i]` is compiled: base register holds &A[0], index register holds `i`, hardware adds them.

Socratic Check: Which Mode for `A[i]`?

Question

Inside a loop, the compiler must generate code for `A[i] = A[i] + 1` where `i` changes every iteration. Which addressing mode should it use for accessing `A[i]` — and why not direct addressing?

- Direct addressing fails: the address field is fixed at compile time, but `A[i]`'s address changes every iteration.
- Indexed/based addressing wins: EA is recomputed from a *register* (`i`) every time, so the same instruction works for every loop iteration.
- This is why indexed addressing exists: it is the hardware primitive underneath every array access in every high-level language.

Autoincrement / Autodecrement Addressing

Idiom: streaming through an array, or push/pop

`ADD R1, (R2)+` $\Rightarrow$ use `[R2]` as EA, *then* increment `R2` by the operand size.

- Register-indirect *plus* an automatic ± 1 (or $\pm$ operand size) on the register, as a side effect of the instruction.
- Autoincrement steps forward (array traversal, stack pop); autodecrement steps backward (stack push).
- No separate "increment" instruction needed — one fewer instruction per loop iteration.

Addressing Modes: Side by Side

Mode	Syntax	EA	Typical use
Immediate	`#5`	— (value in instr.)	Constants
Direct	`1000`	A	Global variables
Indirect	`(1000)`	$M[A]$	Pointers via memory
Register	`R2`	— (value in reg.)	Local variables
Register-indirect	`(R2)`	$[R2]$	Pointer variables
Displacement	`d(R2)`	$[R2] + d$	Array elements, structs
Autoincrement	`(R2)+`	$[R2]$, then $R2{+}{=}n$	Array streaming, stack pop

Every row exists because some code idiom needs it — not to be memorized in isolation.

Worked Example: Computing EA by Hand

Instruction

`LOAD R1, 20(R2)` (displacement addressing), given $[\text{R2}] = 1000$

1. Addressing mode: displacement $\Rightarrow$ EA $= [\text{R2}] + d$.
2. Displacement field: $d = 20$.
3. EA $= 1000 + 20 = 1020$.
4. Operand fetched from $M[1020]$; result placed in R1.

Cost accounting

One register read (to get `[R2]`) + one memory read (to get $M[\text{EA}]$) = cheaper than indirect addressing's two *memory* reads.

Addressing modes tell the CPU WHERE. Now: what hardware moves the data?

From “Where” to “How”

The next question

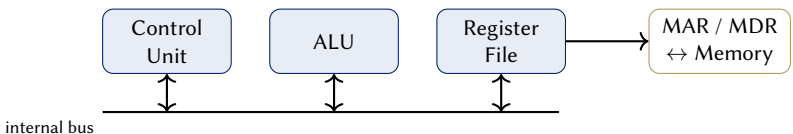
An instruction like ADD R1, R2, R3 needs a register file, an ALU, and a *path* connecting them so operands can travel from registers to the ALU and results can travel back. How much wiring do we build — and what does that choice cost us?

Two architectural choices dominate this design space:

- **Single-bus organization** — one shared path, everything takes turns.
- **Multiple-bus organization** — several parallel paths, more can happen per clock cycle.

We will build both, trace the same instruction through each, and count cycles.

The CPU's Internal Building Blocks



Everything on the next several slides zooms into the box you have not seen yet: *how the register file, ALU, and buses actually wire together.*

Notation: Register Transfer Language (RTL)

RTL notation (added to the course registry)

- R1 $\leftarrow$ [R2] reads as: "the contents of R2 are transferred into R1."
- [X] always means *contents of* X (a register or memory cell).
- M[EA] means the memory cell at address EA.
- A **control step** (written $T_1, T_2, \ldots$) is one clock cycle's worth of RTL transfers.

We will use RTL to trace instructions through hardware for the rest of this lecture, and again in Week 6 when we build the control unit that *generates* these $T_1, T_2, \ldots$ sequences automatically.

Socratic Check: One Bus, or Many?

Design question

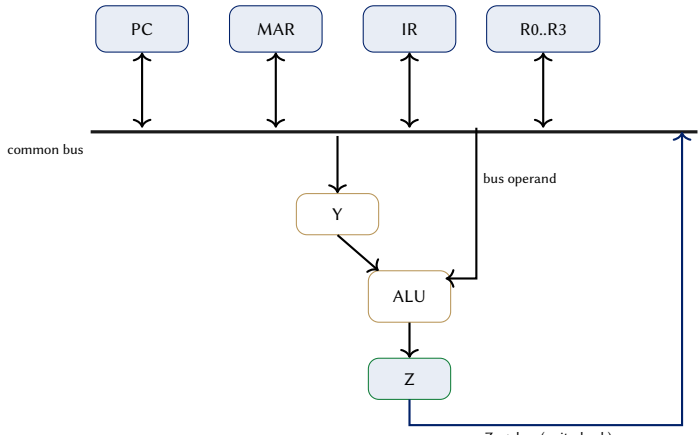
`ADD R1, R2, R3` needs two register values to reach the ALU and one result to come back. If every register and the ALU share a *single* path, can two register values reach the ALU in the same clock cycle?

- If the answer is no, the instruction needs *multiple* clock cycles just to move data around — before the ALU even computes anything extra.
- If we instead give the ALU two dedicated input paths, one instruction could complete data movement in a single cycle — at the cost of more wires.

Let's build both and count.

Design 1: everything shares ONE bus.

The Single-Bus Datapath



[2] — cf. Fig. 5-4, p.129 (registers and memory on the common bus system); the Y/Z buffer registers shown here are this course's pedagogical simplification of the ALU operand path.

How the Single Bus Works

The core constraint

Only *one* value can be on the bus at any instant. Every transfer between registers happens *through* the bus, one at a time.

- The ALU needs *two* inputs, but the bus can only deliver one value per cycle — so one operand must be parked somewhere while the second arrives. That is what the hidden register Y is for.
- Z holds the ALU's result until it can be written back onto the bus (the bus is busy carrying the second operand at the moment the ALU computes).
- Consequence: a two-operand operation takes *multiple* control steps, not one.

Worked Trace: R1 ← R2 + R3 on a Single Bus

Step	RTL	On the bus	What happens
T_1	$Y \leftarrow [R2]$	$[R2]$	R2 onto bus, latched into Y
T_2	$Z \leftarrow [Y] + [R3]$	$[R3]$	R3 onto bus $\rightarrow$ ALU; ALU adds Y; result $\rightarrow Z$
T_3	$R1 \leftarrow [Z]$	$[Z]$	Z onto bus, latched into R1

Running total

3 control steps for one ADD on the single-bus organization. Keep this number — we compare it to the multi-bus trace shortly.

Why Y and Z Exist

- Y exists because the bus can carry only one operand per cycle, but the ALU needs two simultaneously — Y *remembers* the first one while the second is still in transit.
- Z exists because the ALU's result cannot be written back to the bus in the *same* cycle the bus is occupied by the second operand — Z *holds* the result for one extra step.
- Both are the direct hardware cost of sharing one bus: every two-operand instruction pays for buffering.

Single-Bus Organization: Trade-offs

Cost vs. speed

- Pro: one bus = one set of wires and multiplexers — cheapest to build, easiest to extend with new registers.
- Pro: simple, uniform control logic (every transfer looks the same).
- Con: every multi-operand instruction takes several control steps — more clock cycles per instruction, so lower throughput.
- Con: needs the extra Y/Z buffering hardware around the ALU.

Socratic Check: Counting Cycles

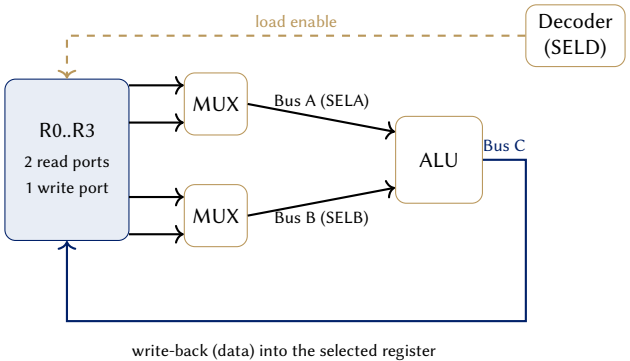
Question

If every bus transfer takes exactly one clock cycle, how many cycles does $\mathrm{R1} \leftarrow \mathrm{R2} + \mathrm{R3}$ take on the single-bus organization we just traced?

- Count the control steps in the trace: T_1, T_2, T_3.
- **Answer: 3 cycles** — one to stage the first operand, one to stage the second *and* compute, one to write back the result.
- Hold onto this number. We are about to see a design that does the same job in one cycle.

Design 2: give the ALU dedicated inputs.

The Three-Bus Datapath



Bus A and Bus B read two *different* registers in the *same* cycle (via MUX-selected SELA/SELB); **Bus C** carries the ALU's result directly back into the register file, while the SELD decoder (dashed) independently drives that register's load-enable line — cf. Mano, Fig. 8-2, p.243 ("Registers with common ALU"); the same three-bus principle is also presented in Hamacher Ch. 7 [2, 1].

Worked Trace: R1 ← R2 + R3 on Three Buses

Step	RTL	On the buses	What happens
T_1	$R1 \leftarrow [R2] + [R3]$	A=$[R2]$, B=$[R3]$, C=result	Both operands read *and* the ALU output written back, all in one cycle

Running total

1 control step for the same ADD — versus **3** on the single-bus organization. Same instruction, same ALU, different wiring.

Why Three Buses Are Faster

- Bus A and Bus B are *separate physical paths* — the register file can put two different registers' contents on them *simultaneously*.
- The ALU no longer needs a Y buffer: both inputs arrive in the same cycle, directly from the register file.
- Bus C lets the result be written back *in the same cycle* it is computed, since it is not competing with Bus A/B for the same wire.
- Net effect: the sequential bottleneck from the single-bus design simply does not exist here.

Three-Bus Organization: Trade-offs

Cost vs. speed

- Pro: most two-operand register instructions complete data movement in a single control step.
- Pro: no Y/Z buffering needed for the common case.
- Con: three times the bus wiring, plus the multiplexers needed to route any register onto any of the three buses.
- Con: more complex control logic — three buses must be driven correctly every cycle, not just one.

Single-Bus vs. Three-Bus: The Full Comparison

	Single bus	**Three bus**
Wiring cost	Low (one bus)	High (three buses + muxes)
Cycles for $R1 \leftarrow R2+R3$	3 (T_1, T_2, T_3)	1 (T_1)
Extra registers needed	Y, Z (buffering)	None (common case)
Control logic	Simple, uniform	More complex, more signals
Best fit	Cost-sensitive designs	Performance-sensitive designs

Neither design is "correct" — they sit at different points on the same cost/speed trade-off curve.

Socratic Check: Why Not *N* Buses?

Question

If three buses beat one, why not build a CPU with ten buses, or a hundred?

- Every additional bus needs its own wiring *and* its own multiplexer logic at every register — cost grows roughly with the number of buses, not for free.
- Most instructions only need two operand reads and one result write — beyond three buses, there is little left to parallelize *within a single instruction*.
- Real speedups beyond this point come from a different idea entirely: *overlapping different instructions* in time — which is exactly what pipelining (Week 11) does.

The Two Threads Meet

Addressing modes have a bus-organization cost too

Recall indirect addressing: *two* memory reads to resolve EA. On a single-bus CPU, each of those reads is itself another multi-step bus transaction — the "expensive" addressing modes and the "expensive" bus organization compound.

- Cheap addressing (immediate, register) + cheap bus (single) = minimal hardware, more cycles per instruction.
- Expensive addressing (indirect) + expensive bus (three-bus) can still be fast, but every design choice is paid for in wires or in cycles — never free.

Bridge to Week 6

What we built today

- Seven addressing modes, each motivated by a real code idiom.
- A single-bus datapath, traced step by step: 3 control steps for R1 $\leftarrow$ R2+R3.
- A three-bus datapath, same instruction: 1 control step.

What's still missing

We wrote T_1, T_2, T_3 by hand. **Something has to generate that sequence automatically**, for *every* instruction in the ISA, every time it runs. That "something" is the **control unit** — hardwired control, next week.

Summary

- **Addressing mode** = the rule for computing EA from an instruction's address field; each mode trades flexibility against extra memory/bus reads.
- **RTL notation** ($R \leftarrow [\cdot]$, $M[\text{EA}]$, control steps $T_1, T_2, \ldots$) is how we describe hardware-level data movement precisely.
- **Single-bus organization**: cheap wiring, sequential transfers, needs Y/Z buffering, more cycles per instruction.
- **Multiple-bus organization**: parallel register reads and writes, fewer cycles per instruction, more wiring and control complexity.
- Same instruction, same ALU, different wiring $\Rightarrow$ 3 cycles vs. 1 — the cost/speed trade-off, made concrete.

References

V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.
Computer Organization.
McGraw-Hill, 5th edition, 2002.

M. Morris Mano.
Computer System Architecture.
Prentice-Hall, 3rd edition, 1993.